Pasos para configurar el entorno – MotoService

Evidencia de configuración · Web Services · Integrante: German Moreno
Equipo: macOS (Apple Silicon) · terminal zsh · Python 3.13.13 · fecha: 04/11/2026

Seguí los mismos pasos de la guía del profesor y de la guía del equipo. Como trabajo en Mac y no en Windows, algunos comandos cambian (activar el venv, copiar archivos); abajo de cada paso dejo la salida real que me dio la terminal.

Antes de empezar: clonar el repositorio y pasar a develop

```
git clone https://github.com/notkahuate/Sistemas-para-taller-de-Mecanica.git
cd Sistemas-para-taller-de-Mecanica
git checkout develop
```

El código está en la rama **develop** (en main solo está el README). *git checkout develop* no crea una rama nueva: usa la que ya existe en GitHub.

Resultado obtenido en mi equipo:

```
$ git checkout develop
Switched to a new branch 'develop'
branch 'develop' set up to track 'origin/develop'.
$ git branch -a
* develop
  main
  remotes/origin/HEAD -> origin/main
  remotes/origin/develop
  remotes/origin/main
  remotes/origin/staging
  remotes/origin/testing
```

Paso 01 – Verificar Python

Se necesita 3.10 o mayor. El Python que trae el Mac es 3.9.6, así que instalé Python 3.13 (la misma versión de la sala) con **uv**:

```
python3 --version
uv python install 3.13
python3.13 --version
```

Resultado obtenido en mi equipo:

```
$ python3 --version
Python 3.9.6
$ uv python install 3.13
Installed Python 3.13.13 in 3.95s
 + cpython-3.13.13-macos-aarch64-none (python3.13)
$ python3.13 --version
Python 3.13.13
```

Paso 02 – Crear el entorno virtual

```
python3.13 -m venv venv
```

Resultado obtenido en mi equipo:

```
$ python3.13 -m venv venv
$ ls venv
bin
include
lib
pyvenv.cfg
```

Paso 03 – Activar el entorno virtual

En Mac no hace falta Set-ExecutionPolicy; se activa con *source*:

```
source venv/bin/activate
```

La terminal queda con (venv) y *python* apunta al del proyecto:

Resultado obtenido en mi equipo:

```
$ source venv/bin/activate
$ which python
/Volumes/Kingston/motoservices/Sistemas-para-taller-de-Mecanica/venv/bin/python
$ python --version
Python 3.13.13
```

Paso 04 – Instalar dependencias

```
python -m pip install --upgrade pip
pip install -r requirements.txt
pip list
```

Resultado obtenido en mi equipo:

```
$ python -m pip install --upgrade pip
Successfully installed pip-26.3
$ pip install -r requirements.txt
Successfully installed Mako-1.4.3 MarkupSafe-3.0.4 alembic-1.13.1 annotated-doc-0.0.5 annotated-types-0.8.0
$ pip list
Package           Version
alembic           1.13.1
fastapi           0.111.0
pydantic          2.9.2
python-dotenv     1.0.1
SQLAlchemy        2.0.30
uvicorn           0.30.0
```

Aparecen fastapi, uvicorn, sqlalchemy, pydantic 2.9.2, python-dotenv y alembic.

Paso 05 – Revisar la estructura

Se ve app/ con api, domain, services, repositories y core, además de main.py y requirements.txt (.env.example también está; ls no lo muestra porque empieza con punto).

Resultado obtenido en mi equipo:

```
$ ls
README.md
app
main.py
requirements.txt
tests
venv
$ ls app
__init__.py
api
core
domain
repositories
services
```

Paso 06 – Variables de entorno

```
cp .env.example .env     # en Windows: copy
```

Resultado obtenido en mi equipo:

```
$ cp .env.example .env
$ git check-ignore .env
.env
```

git check-ignore confirma que .env está en el .gitignore, o sea que no se sube a GitHub.

Paso 07 – Ejecutar el servidor

Primero lo intenté en el puerto 8000, como dice la guía, y falló porque ese puerto ya lo usaba Docker (otro proyecto en mi equipo):

Resultado obtenido en mi equipo:

```
$ uvicorn main:app --reload --host 0.0.0.0 --port 8000
INFO:     Will watch for changes in these directories: ['/Volumes/Kingston/motoservices/Sistemas-para-taller-de-
    Mecanica']
ERROR:    [Errno 48] Address already in use
```

Lo levanté en el puerto 8001:

```
uvicorn main:app --reload --host 0.0.0.0 --port 8001
```

Resultado obtenido en mi equipo:

```
$ uvicorn main:app --reload --host 0.0.0.0 --port 8001
INFO:     Will watch for changes in these directories: ['/Volumes/Kingston/motoservices/Sistemas-para-taller-de-
    Mecanica']
INFO:     Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [10497] using WatchFiles
INFO:     Started server process [10503]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:51973 - "GET /health HTTP/1.1" 200 OK
```

Paso 08 – Verificar en el navegador

Abrí http://localhost:8001/docs. En GET /health hice clic en "Try it out" y luego en "Execute". Respondió código 200 con status "ok".

Resultado obtenido en mi equipo:

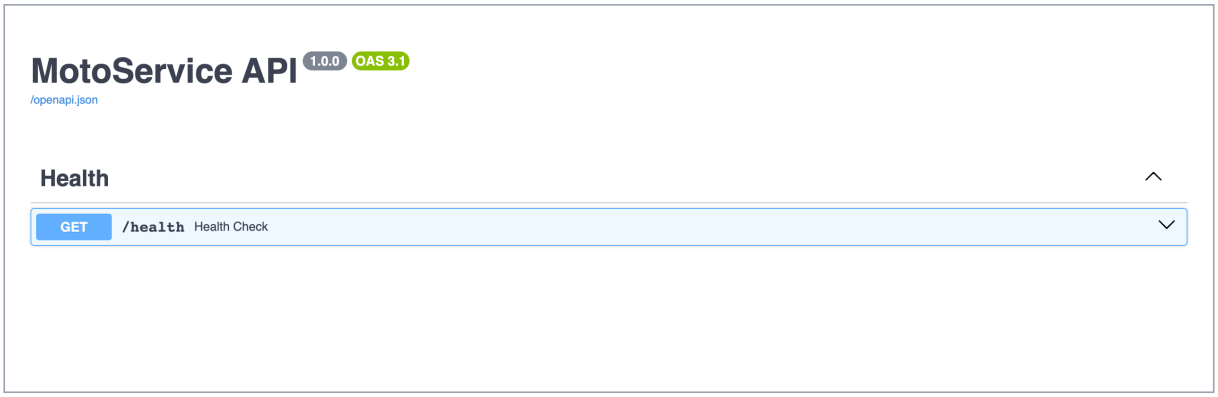
```
$ curl -i http://localhost:8001/health
HTTP/1.1 200 OK
date: Wed, 04 Nov 2026 14:11:26 GMT
server: uvicorn
content-length: 61
content-type: application/json

{"status":"ok","version":"1.0.0","environment":"development"}

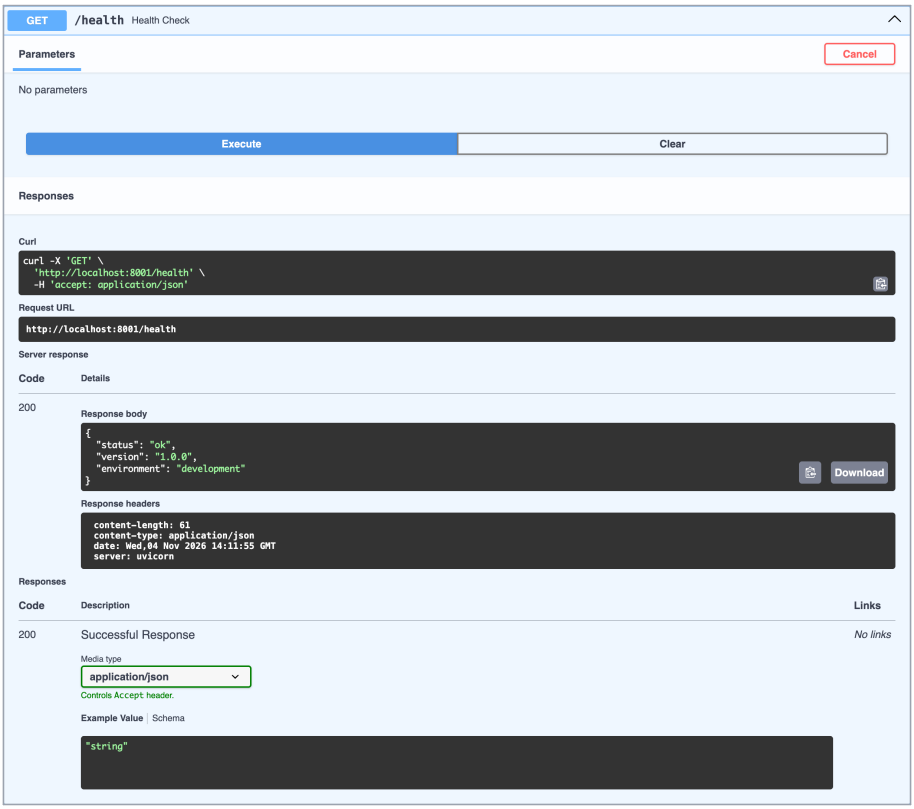
$ curl -s -o /dev/null -w "%{http_code}" http://localhost:8001/docs
200
```

Capturas

1. Swagger en localhost:8001/docs:



2. GET /health ejecutado (código 200 y respuesta):



Problemas que encontré y cómo los resolví

• **Python 3.9.6 en el Mac**: es menor que 3.10. Instalé Python 3.13 con *uv python install 3.13* y creé el venv con python3.13.

• **[Errno 48] Address already in use**: el puerto 8000 lo ocupaba Docker. Usé *--port 8001* (también se puede cerrar lo que usa el 8000; se ve con *lsof -i :8000*).

• **Comandos de Windows**: en Mac: *source venv/bin/activate* en vez de Activate.ps1 y *cp* en vez de *copy*.

Al terminar, detuve el servidor con Ctrl + C.